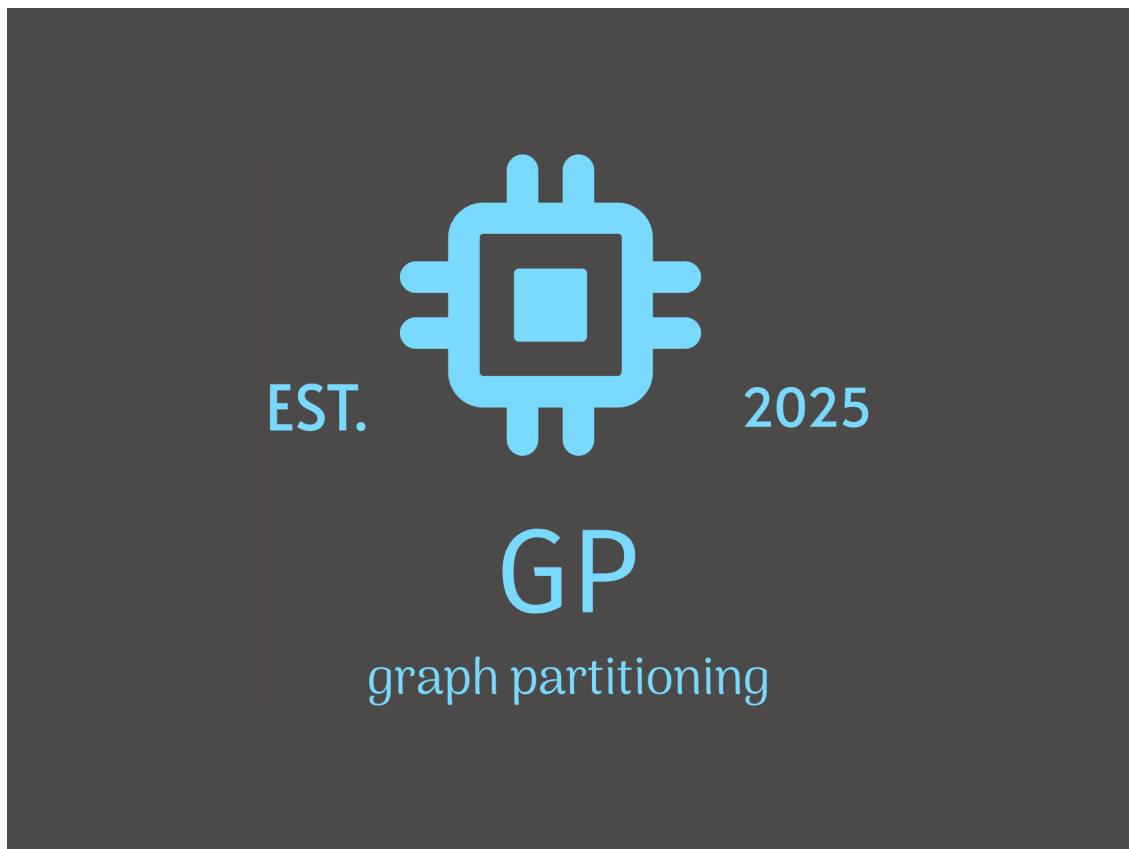


Dokumentacja końcowa projektu
Aplikacja do obsługi i podziału grafów

Wojciech Ziembowicz, Damian Bródkowski

3 czerwca 2025



Spis treści

1	Wprowadzenie	3
2	Instrukcja obsługi aplikacji	3
2.1	Zasada działania programu	3
2.2	Podstawowe funkcje	3
3	Algorytm podziału spektralnego	4
4	Struktura projektu	5
4.1	Główne pakiety	5
4.2	Główne klasy	5

5 Wybrane fragmenty kodu	6
5.1 System wielojęzyczności	6
5.2 Dynamiczne ładowanie instrukcji	7
5.3 Wizualizacja podziału grafu	7
5.4 Zapisywanie wyników podziału	8
6 Interfejs graficzny użytkownika	9
6.1 Główne okno aplikacji	9
6.2 Przykład wizualizacji podziału grafu	10
6.3 Menu i zmiana języka	10
6.4 Wyświetlenie instrukcji obsługi	11
7 Formaty plików	11
7.1 Formaty plików wejściowych	11
7.2 Formaty plików wyjściowych	12
8 Mechanizm parsowania plików wejściowych	14
9 Wizualizacja i nawigacja po grafie	15
10 Diagram klas	18
11 Wzorce projektowe	19
12 Instalacja i wymagania systemowe	19
13 Podsumowanie i możliwości rozbudowy	19

1 Wprowadzenie

Celem projektu jest stworzenie aplikacji desktopowej do wizualizacji, dzielenia oraz analizy grafów z interfejsem graficznym napisanym w Javie (Swing). Program pozwala użytkownikowi ładować grafy z plików tekstowych, konfigurować parametry podziału oraz uzyskiwać wyniki wizualne i tekstowe podziału grafu na podgrafy.

Aplikacja implementuje algorytm podziału spektralnego (spectral partitioning), który jest uznawany za jedną z najbardziej efektywnych metod dzielenia grafów. Podejście to wykorzystuje właściwości wektora Fiedlera (drugiego najmniejszego wektora własnego macierzy Laplace’a grafu) do wyznaczenia optymalnego podziału.

Projekt obsługuje wielojęzyczność (polski, angielski), posiada wbudowaną czytelnie sformatowaną instrukcję oraz nowoczesny interfejs z możliwością interaktywnej nawigacji po grafie.

2 Instrukcja obsługi aplikacji

2.1 Zasada działania programu

Po uruchomieniu programu użytkownik widzi główne okno z wizualizacją przykładowego grafu (domyślnie prosty trójkąt), suwakami do ustawiania parametrów (próg marginesu i liczba podgrafów) oraz przyciskami do dzielenia oraz resetowania grafu.

Program oferuje intuicyjną nawigację po grafie przy użyciu myszy:

- Przesuwanie widoku (przeciąganie)
- Przybliżanie/oddalanie (rolka myszy)
- Automatyczne dopasowanie widoku do wielkości okna

2.2 Podstawowe funkcje

- **Wczytywanie grafu:** Wczytywanie następuje z plików w formatach CSRRG lub tekstowych zgodnych z opisanym w instrukcji formatem. Obsługiwane są również pliki binarne dla zapisanych wcześniej podziałów.
- **Podział grafu:** Po ustawieniu parametrów użytkownik może podzielić graf na zadane podgrafy za pomocą przycisku “Podziel”. Algorytm spektralny oblicza najlepszy podział grafu uwzględniając:
 - Liczbę części, na które ma być podzielony graf (ustawienie suwaka)
 - Margines błędu określający dopuszczalną różnicę wielkości poszczególnych części
- **Resetowanie:** Przycisk “Reset” przywraca początkowy, przykładowy graf i domyślne parametry.
- **Instrukcja:** Aplikacja posiada wbudowaną instrukcję, wywoływaną przez menu (sekcja 6.4), automatycznie dostosowującą się do wybranego języka.
- **Zmiana języka:** Możliwa z poziomu menu. Interfejs automatycznie dostosowuje napisy do wybranego języka dzięki systemowi obsługi wielu wersji językowych.

- **Zapis podziału:** Dostępny z poziomu menu, w formatach BIN oraz TXT, z zachowaniem pełnej informacji o strukturze grafu i jego podziale.

3 Algorytm podziału spektralnego

Aplikacja wykorzystuje zaawansowany algorytm podziału spektralnego (spectral partitioning), który jest implementowany w klasie `PartitionAlg`. Główne etapy algorytmu:

1. Obliczenie macierzy Laplace'a grafu ($L = D - A$, gdzie D to macierz stopni, a A to macierz sąsiedztwa)
2. Znalezienie drugiego najmniejszego wektora własnego (wektora Fiedlera) tej macierzy
3. Sortowanie wierzchołków według wartości w wektorze Fiedlera
4. Podział grafu na podstawie posortowanych wierzchołków
5. Optymalizacja punktu podziału dla minimalizacji liczby przeciętych krawędzi

Warto podkreślić, że implementacja algorytmu została zoptymalizowana, aby działać efektywnie nawet dla dużych grafów:

```

1 private static double[] computeFiedlerVectorImplicit(Graph graph, int V)
2 {
3     double[] x = new double[V];
4     double[] y = new double[V];
5     double[] prevX = new double[V];
6
7     // Inicjalizacja wektora losowymi wartościami
8     Random rand = new Random();
9     for (int i = 0; i < V; i++) {
10         x[i] = rand.nextDouble() - 0.5;
11     }
12
13     // Usunięcie składowej stałej (ortogonalizacja do pierwszego
14     // wektora własnego)
15     double sum = 0;
16     for (int i = 0; i < V; i++) sum += x[i];
17     double avg = sum / V;
18     for (int i = 0; i < V; i++) x[i] -= avg;
19
20     // Normalizacja wektora startowego
21     double norm = norm(x, V);
22     for (int i = 0; i < V; i++) x[i] /= norm;
23
24     // Iteracja potęgowa z deflacją
25     int maxIter = 300;
26     double EPSILON = 1e-10;
27     double lambda = 0.0;
28
29     for (int iter = 0; iter < maxIter; iter++) {
30         // Zachowujemy poprzedni wektor x do sprawdzenia zbieżności
31         System.arraycopy(x, 0, prevX, 0, V);
32
33         // Mnożenie macierzy "w locie" bez jej tworzenia

```

```

32     implicitLaplacianMultiply(graph, x, y, V);
33
34     // Usuni cie komponentu wzd u pierwszego wektora w asnego
35     sum = 0;
36     for (int i = 0; i < V; i++) sum += y[i];
37     avg = sum / V;
38     for (int i = 0; i < V; i++) y[i] -= avg;
39
40     // Obliczenie warto ci w asnej (przybli enie)
41     double dotProduct = 0;
42     for (int i = 0; i < V; i++) {
43         dotProduct += x[i] * y[i];
44     }
45     lambda = dotProduct;
46
47     // Normalizacja
48     norm = norm(y, V);
49     if (norm < EPSILON) break;
50
51     for (int i = 0; i < V; i++) x[i] = y[i] / norm;
52
53     // Sprawdzenie zbie no ci
54     double diff = 0;
55     for (int i = 0; i < V; i++) {
56         diff += Math.abs(x[i] - prevX[i]);
57     }
58
59     if (diff < EPSILON) break;
60 }
61
62 return x;
63 }

```

Listing 1: Fragment algorytmu obliczającego wektor Fiedlera bez tworzenia pełnej macierzy

4 Struktura projektu

4.1 Główne pakiety

Projekt został zorganizowany w modułową strukturę zawierającą następujące pakiety:

- `com.example.model` – klasy modelu danych (`Graph`, `Group`, `PartitionAlg`)
- `com.example.ui` – komponenty interfejsu użytkownika
- `com.example.utils` – klasy narzędziowe (parsery, system wielojęzyczności)
- `resources` – zasoby aplikacji (pliki językowe, instrukcje)

4.2 Główne klasy

- **AppFrame** – główna ramka aplikacji; obsługuje menu, zarządza głównym widokiem, otwiera instrukcję obsługi i zapewnia dostęp do podstawowych funkcji.

- **MainView** – panel z główną logiką interfejsu użytkownika (suwaki, przyciski, wizualizacja grafu). Odpowiada za interakcję z użytkownikiem i kontrolę głównych parametrów podziału.
- **GraphPanel** – komponent odpowiedzialny za renderowanie grafu i jego podziałów. Implementuje zaawansowane funkcje nawigacji jak przybliżanie/oddalanie i przesuwanie widoku.
- **GraphPostPartitionPanel** – wyspecjalizowany panel do wyświetlania grafu po podziale, zapewniający kolorowe oznaczenie komponentów podgrafu.
- **Graph** – model grafu przechowujący dane o wierzchołkach i krawędziach w formie listy sąsiedztwa (adjacency list).
- **PartitionAlg** – implementacja algorytmu spektralnego do podziału grafu, wykorzystująca zaawansowane metody numeryczne.
- **Group** – klasa reprezentująca grupę (podgraf) powstałą w wyniku podziału.
- **TXTParser** / **BINParser** / **CSRParser** – klasy odpowiedzialne za wczytywanie różnych formatów plików.
- **FileSaver** – klasa odpowiedzialna za zapisywanie wyników podziału grafu do plików.
- **LanguageManager** – obsługa wielojęzyczności i tłumaczeń. System bazuje na plikach zasobów i pozwala na dynamiczną zmianę języka.
- **ManualLoader** – ładowanie instrukcji obsługi w wybranym języku z plików Markdown.

5 Wybrane fragmenty kodu

5.1 System wielojęzyczności

Jedną z zaawansowanych funkcji aplikacji jest system wielojęzyczności, zaimplementowany w klasie `LanguageManager`:

```

1 public class LanguageManager {
2     private static Locale currentLocale = Locale.forLanguageTag("pl");
3     private static ResourceBundle bundle = ResourceBundle.getBundle("
    Messages", currentLocale);
4
5     public static void setLanguage(Locale locale) {
6         currentLocale = locale;
7         bundle = ResourceBundle.getBundle("Messages", currentLocale);
8     }
9
10    public static String get(String key) {
11        return bundle.getString(key);
12    }
13
14    public static Locale getCurrentLocale() {
15        return currentLocale;
16    }

```

17 }

Listing 2: Implementacja systemu wielojęzyczności

System ten pozwala na łatwe tłumaczenie interfejsu i instrukcji bez konieczności modyfikacji kodu aplikacji.

5.2 Dynamiczne ładowanie instrukcji

Aplikacja posiada wbudowaną instrukcję obsługi w formacie Markdown, która jest dynamicznie ładowana i wyświetlana w języku wybranym przez użytkownika:

```

1 public static String loadManual(String language) {
2     String manualPath = "/manual/manual_" + language + ".md";
3     try (InputStream is = ManualLoader.class.getResourceAsStream(
4         manualPath);
5         BufferedReader reader = new BufferedReader(new
6             InputStreamReader(is, StandardCharsets.UTF_8))) {
7         return reader.lines().collect(Collectors.joining("\n"));
8     } catch (IOException | NullPointerException e) {
9         System.err.println("Nie można wczytać instrukcji obsługi: " +
10             e.getMessage());
11         return "Instrukcja obsługi nie jest dostępna.";
12     }
13 }
```

Listing 3: Kod ładowania instrukcji obsługi

5.3 Wizualizacja podziału grafu

Klasa GraphPostPartitionPanel implementuje zaawansowany system wizualizacji podziału grafu, wykorzystując techniki kolorowania dla wyraźnego oznaczenia komponentów:

```

1 // Kolory dla komponentów (max 10 kolorów, potem się powtarzają)
2 private static final Color[] COMPONENT_COLORS = {
3     new Color(64, 8, 175), // fioletowy
4     new Color(255, 100, 224), // różowy
5     new Color(100, 255, 100), // zielony
6     new Color(255, 200, 100), // pomarańczowy
7     new Color(200, 100, 255), // fioletowy
8     new Color(100, 255, 255), // cyjan
9     new Color(255, 100, 255), // magenta
10    new Color(255, 255, 100), // żółty
11    new Color(150, 150, 150), // szary
12    new Color(100, 255, 200) // miętowy
13 };
14
15 // Zwraca kolor dla danego komponentu
16 private Color getComponentColor(int componentIndex) {
17     return COMPONENT_COLORS[componentIndex % COMPONENT_COLORS.length];
18 }
```

Listing 4: Fragment kodu odpowiedzialny za kolorowanie komponentów grafu

5.4 Zapisywanie wyników podziału

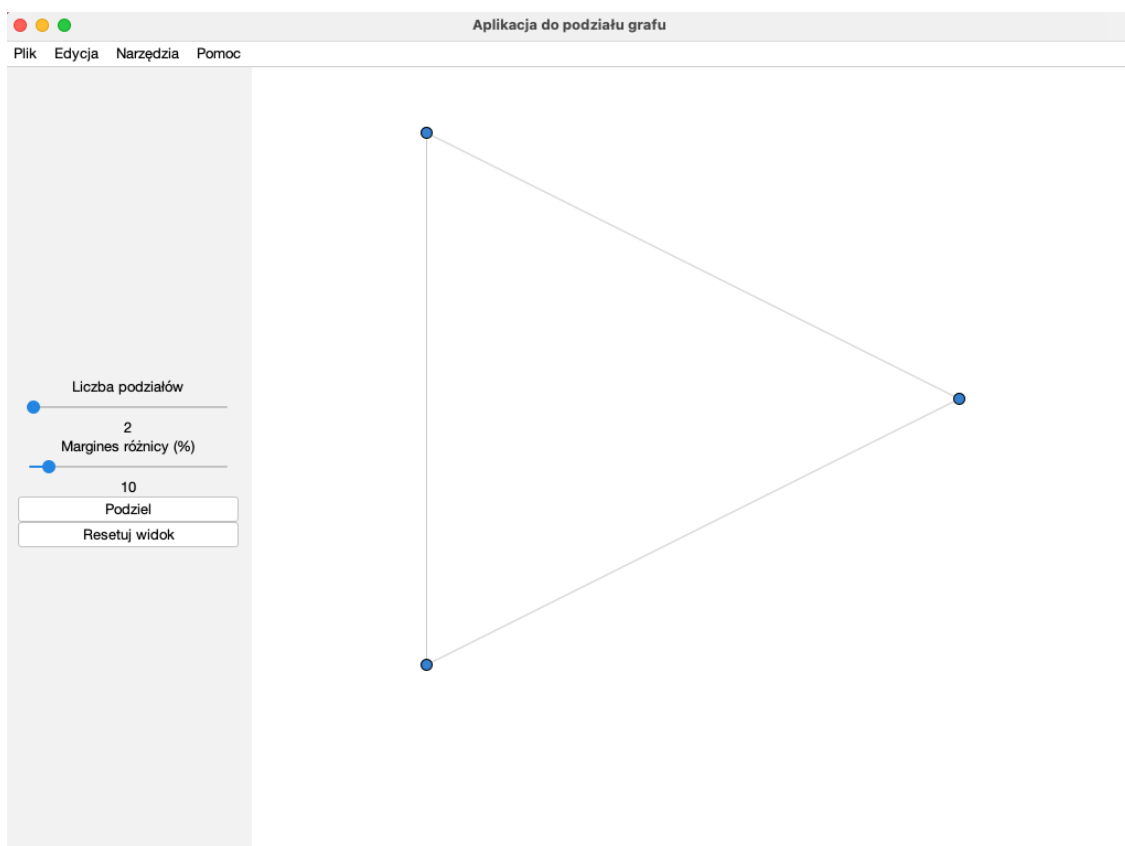
Aplikacja umożliwia zapisywanie wyników podziału grafu w formatach TXT i BIN. Poniżej fragment klasy FileSaver odpowiedzialnej za tę funkcjonalność:

```
1 public static void saveTxtFormat(File file, int vertexCount, List<
    Integer> adjacencyList,
2                                     List<Integer> adjacencyIndices, Map<
    Integer, Integer> partitionData,
3                                     boolean hasPartition) throws IOException
4 {
5     try (PrintWriter writer = new PrintWriter(new FileWriter(file))) {
6         // Utworzenie macierzy s siedztwa
7         int[][] adjacencyMatrix = new int[vertexCount][vertexCount];
8
9         // Wype nienie macierzy s siedztwa
10        for (int i = 0; i < vertexCount; i++) {
11            if (i >= adjacencyIndices.size() - 1) continue;
12
13            int startIdx = adjacencyIndices.get(i);
14            int endIdx = adjacencyIndices.get(i + 1);
15
16            for (int j = startIdx; j < endIdx; j++) {
17                if (j >= adjacencyList.size()) continue;
18
19                int neighbor = adjacencyList.get(j);
20                adjacencyMatrix[i][neighbor] = 1;
21            }
22
23            // Zapisanie macierzy s siedztwa
24            for (int i = 0; i < vertexCount; i++) {
25                for (int j = 0; j < vertexCount; j++) {
26                    writer.print(adjacencyMatrix[i][j]);
27                    if (j < vertexCount - 1) writer.print(" ");
28                }
29                writer.println();
30            }
31
32            // Dodanie pustej linii po macierzy
33            writer.println();
34
35            // Zapisywanie informacji o podziale
36            if (hasPartition) {
37                // Kod obs uguj cy zapisywanie grup...
38            }
39        }
40    }
```

Listing 5: Fragmenty kodu zapisującego wyniki podziału grafu

6 Interfejs graficzny użytkownika

6.1 Główne okno aplikacji



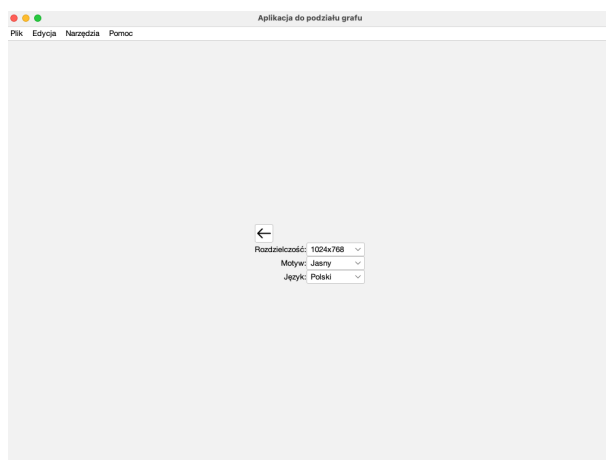
Rys. 1: Główne okno programu z interfejsem zawierającym panel grafu, suwaki do kontroli parametrów podziału oraz przyciski funkcyjne.

6.2 Przykład wizualizacji podziału grafu



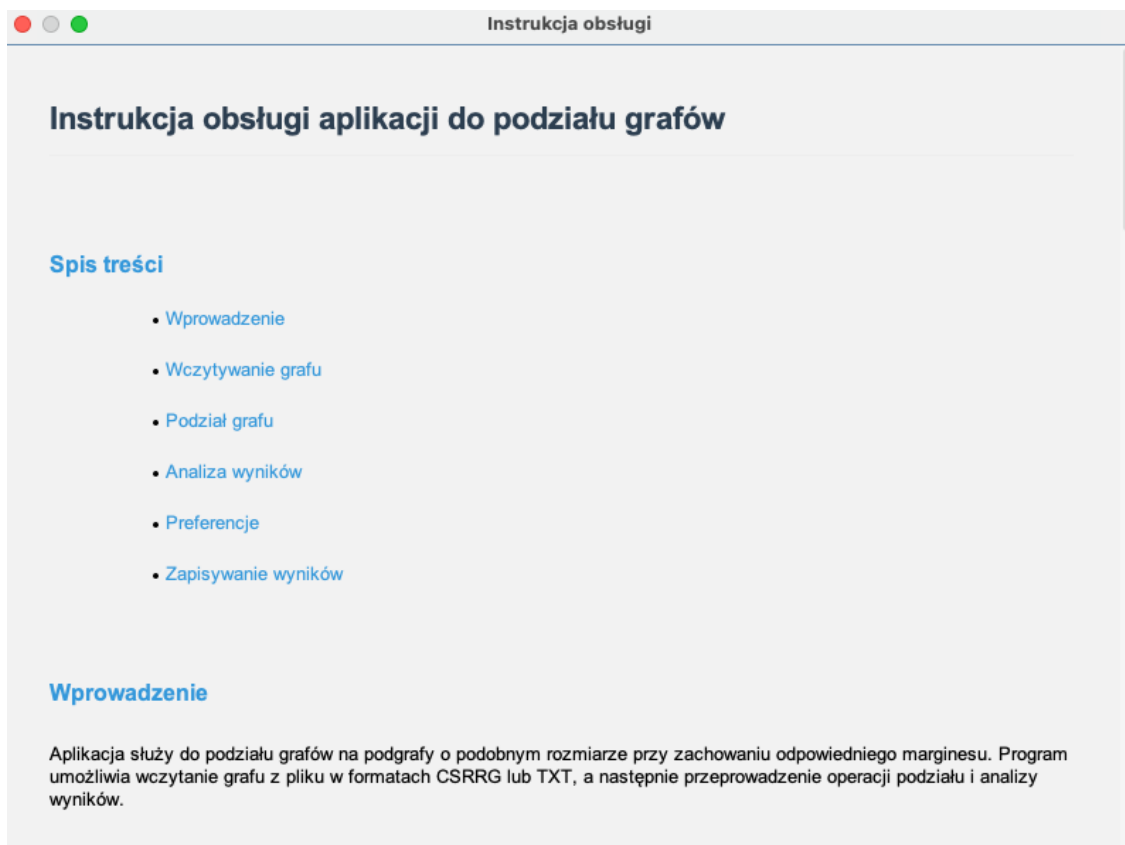
Rys. 2: Wizualizacja grafu po podziale na dwie grupy. Wierzchołki i krawędzie są kolorowane zgodnie z przypisanymi grupami, co zapewnia przejrzystą prezentację wyników podziału.

6.3 Menu i zmiana języka



Rys. 3: Menu aplikacji z rozwiniętą opcją zmiany języka. Aplikacja obsługuje język polski i angielski z możliwością rozszerzenia o kolejne języki.

6.4 Wyświetlenie instrukcji obsługi



Rys. 4: Okno instrukcji obsługi z dokumentacją w formacie Markdown, automatycznie wyświetlanej w aktualnie wybranym języku.

7 Formaty plików

7.1 Formaty plików wejściowych

Aplikacja obsługuje trzy główne formaty plików wejściowych:

a) **Format CSR** – format standardowy do reprezentacji grafów, zawierający:

- Liczbę wierzchołków
- Listę wierzchołków
- Indeksy rozmieszczenia wierzchołków
- Listę sąsiedztwa
- Indeksy listy sąsiedztwa

Przykład pliku CSR:

```
1 105
2 0;1;2;3;4;5;6;7;8;9;10;11;12;13;14;15;16;17;18;19;20;21;22;23;24;25;26;27;28;29
3 0;1;2;3;4;5;6;7;8;9;10;11;12;13;14;15;16;17;18;19;20;21;22;23;24;25;26;27;28;29
```

```

4 4;39;72;91;76;79;80;42;100;103;23;29;57;58;75;82;67;68;72;103;69;89;97;89;47;60
5 0;4;8;12;16;18;21;25;27;31;37;40;45;51;56;59;63;68;72;75;77;81;84;87;90;95;97;1
6

```

Listing 6: Przykładowy plik CSRRG

b) **Format TXT** – format tekstowy zawierający:

- Macierz sąsiedztwa (wartości 0/1 oddzielone spacjami)
- Pustą linię oddzielającą
- Opcjonalnie: definicje grup w formacie "grupa X" z listą krawędzi w formacie "a-b"

Przykład pliku TXT:

```

1 0 1 1
2 1 0 1
3 1 1 0
4
5 grupa 0
6 0-1
7 0-2
8
9 grupa 1
10 1-2
11

```

Listing 7: Przykładowy plik TXT

c) **Format BIN** – binarny format przechowujący:

- Liczbę wierzchołków (int)
- Liczbę grup (int)
- Rozmiar listy sąsiedztwa (int)
- Rozmiar tablicy indeksów (int)
- Listę sąsiedztwa (tablica int)
- Tablicę indeksów (tablica int)
- Dane grup (ID grupy, liczba krawędzi, krawędzie)

7.2 Formaty plików wyjściowych

Aplikacja pozwala na zapisywanie wyników podziału grafu w dwóch formatach:

a) **Format TXT** – plik tekstowy zawierający:

- Macierz sąsiedztwa reprezentującą graf
- Sekcje dla każdej grupy z listą krawędzi należących do tej grupy

Strukturę pliku wyjściowego przedstawia poniższy przykład:

```

1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
2 0 0 0 1 0 1 1 0 0 1 1 0 0 1 1 1 0 0
3 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 1 0 0
4 0 0 0 0 1 1 0 1 1 1 0 0 1 0 1 0 0 0
5 0 0 0 0 1 0 0 0 1 1 1 1 1 1 1 1 0 0
6 0 1 1 1 0 0 0 1 0 1 0 0 0 1 0 1 1 0
7 0 1 0 1 0 1 0 0 0 0 1 1 1 0 0 0 0 0
8 1 0 1 1 0 0 0 1 0 1 0 0 0 1 0 0 0 0
9 0 1 0 0 0 1 1 0 1 0 1 1 1 1 0 1 1 0
10 1 1 0 0 1 1 0 1 1 1 0 1 1 1 0 0 0 0
11 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
12 1 0 1 1 0 1 1 0 0 0 1 1 1 0 0 1 0 0
13 0 0 1 0 0 0 1 0 0 0 0 1 0 1 1 0 1 0
14 0 0 1 1 0 1 1 0 0 1 0 0 1 1 1 0 0 0
15 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
16 0 0 0 1 0 1 0 0 0 1 0 0 1 0 0 0 0 0
17 1 0 1 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0
18 0 0 1 1 0 0 0 1 0 0 1 1 1 0 1 0 0 0
19
20 grupa 0
21 1-47
22 1-71
23 1-101
24 6-16
25 6-93
26 6-98
27 8-28
28 ...
29
30 grupa 1
31 0-4
32 0-39
33 0-72
34 0-91
35 2-5
36 ...
37

```

Listing 8: Przykładowy plik wyjściowy TXT

b) **Format BIN** – plik binarny zawierający te same dane co format TXT, ale w postaci binarnej:

- Liczba wierzchołków (4 bajty, int)
- Liczba grup (4 bajty, int)
- Rozmiar listy sąsiedztwa (4 bajty, int)
- Rozmiar indeksów (4 bajty, int)
- Lista sąsiedztwa (tablica int, każdy po 4 bajty)
- Indeksy (tablica int, każdy po 4 bajty)
- Dane o grupach:
 - ID grupy (4 bajty, int)
 - Liczba krawędzi w grupie (4 bajty, int)
 - Krawędzie (pary int, każdy po 4 bajty)

```

1 public static void saveBinFormat(File file, int vertexCount, List<
    Integer> adjacencyList,
2                                     List<Integer> adjacencyIndices, Map<
    Integer, Integer> partitionData,
3                                     boolean hasPartition) throws IOException
    {
4     try (DataOutputStream dos = new DataOutputStream(new
        FileOutputStream(file))) {
5         // Zapisz liczb wierzchołków w
6         dos.writeInt(vertexCount);
7
8         // Zapisz liczb grup
9         Set<Integer> uniqueGroups = new HashSet<>();
10        if (hasPartition) {
11            uniqueGroups.addAll(partitionData.values());
12        }
13        int groupCount = hasPartition ? uniqueGroups.size() : 0;
14        dos.writeInt(groupCount);
15
16        // Zapisz dane grafu w formacie binarnym
17        dos.writeInt(adjacencyList.size());
18        dos.writeInt(adjacencyIndices.size());
19
20        // Zapisz tablicę adjacencyList
21        for (int neighbor : adjacencyList) {
22            dos.writeInt(neighbor);
23        }
24
25        // Zapisz tablicę adjacencyIndices
26        for (int index : adjacencyIndices) {
27            dos.writeInt(index);
28        }
29
30        // Zapisz informacje o grupach
31        if (hasPartition) {
32            // Kod zapisujący dane o grupach...
33        }
34    }
35 }

```

Listing 9: Kod zapisu w formacie binarnym

8 Mechanizm parsowania plików wejściowych

Aplikacja zawiera wyspecjalizowane parsery dla każdego obsługiwanego formatu pliku. Poniżej znajduje się przykład implementacji parsera CSR RG:

```

1 public class CSR RGParser {
2     private int maxVertices1;
3     private ArrayList<Integer> verticesList2;
4     private ArrayList<Integer> verticesPlacement3;
5     private ArrayList<Integer> adjacencyList4;
6     private ArrayList<Integer> adjacencyIndices5;
7
8     public CSR RGParser(File csrrgFile) throws IOException {
9         try (BufferedReader br = new BufferedReader(new
            InputStreamReader(new FileInputStream(csrrgFile)))) {

```

```

10     String line;
11     int lineNumber = 0;
12     while ((line = br.readLine()) != null) {
13         line = line.trim();
14         if (line.isEmpty()) continue;
15
16         ArrayList<Integer> parsedLine = parseLineToList(line);
17
18         switch (lineNumber) {
19             case 0 -> setMaxVertices1(parsedLine.getFirst());
20             case 1 -> setVerticesList2(parsedLine);
21             case 2 -> setVerticesPlacement3(parsedLine);
22             case 3 -> setAdjacencyList4(parsedLine);
23             case 4 -> setAdjacencyIndices5(parsedLine);
24             default -> System.err.println("Nieoczekiwana liczba
linii w pliku: " + lineNumber);
25         }
26         lineNumber++;
27     }
28
29     // Walidacja danych
30     if (adjacencyIndices5 == null || adjacencyList4 == null) {
31         throw new IllegalStateException("Dane s niekompletne."
);
32     }
33     if (adjacencyIndices5.getLast() > adjacencyList4.size()) {
34         throw new IllegalStateException("Ostatni indeks w
adjacencyIndices przekracza rozmiar adjacencyList.");
35     }
36 }
37
38
39 // Metoda pomocnicza parsuj ca lini tekstu do listy liczb
ca kowitych
40 ArrayList<Integer> parseLineToList(String line) {
41     ArrayList<Integer> parsedLine = new ArrayList<>();
42     String[] tokens = line.split(";");
43     for (String token : tokens) {
44         parsedLine.add(Integer.parseInt(token));
45     }
46     return parsedLine;
47 }
48
49 // gettery i settery...
50 }

```

Listing 10: Implementacja parsera CSRRG

9 Wizualizacja i nawigacja po grafie

Aplikacja oferuje zaawansowane funkcje wizualizacji i nawigacji po grafie, implementowane w klasach `GraphPanel` i `GraphPostPartitionPanel`. Główne funkcje:

- Automatyczne rozmieszczanie wierzchołków przy użyciu algorytmu sprężynowego (force-directed layout)

- Interaktywna nawigacja:
 - Przybliżanie/oddalanie (zoom)
 - Przesuwanie widoku (pan)
 - Automatyczne dopasowanie do ekranu
- Kolorowanie komponentów po podziale grafu
- Dynamiczne dostosowywanie wielkości wierzchołków w zależności od poziomu przybliżenia
- Wyświetlanie etykiet wierzchołków przy wystarczającym poziomie przybliżenia

```

1 private void setupMouseListeners() {
2     // Obsługa przesuwania widoku
3     MouseAdapter ma = new MouseAdapter() {
4         @Override
5         public void mousePressed(MouseEvent e) {
6             lastMousePos = e.getPoint();
7         }
8
9         @Override
10        public void mouseDragged(MouseEvent e) {
11            if (lastMousePos != null) {
12                double dx = e.getX() - lastMousePos.x;
13                double dy = e.getY() - lastMousePos.y;
14                panOffset.setLocation(panOffset.getX() + dx, panOffset.
15                getY() + dy);
16                lastMousePos = e.getPoint();
17                repaint();
18                updateViewport();
19            }
20        }
21
22        @Override
23        public void mouseWheelMoved(MouseWheelEvent e) {
24            // Obsługa przybliżania/oddalania
25            double zoom = Math.pow(1.1, -e.getWheelRotation());
26
27            // Centrowanie przybliżenia na pozycji kursora
28            Point2D mouse = e.getPoint();
29            double x = mouse.getX();
30            double y = mouse.getY();
31
32            double newZoom = zoomLevel * zoom;
33
34            // Ograniczenie minimalnego i maksymalnego przybliżenia
35            if (newZoom < 0.05) newZoom = 0.05;
36            if (newZoom > 20) newZoom = 20;
37
38            // Aktualizacja przesunięcia
39            double zoomRatio = newZoom / zoomLevel;
40            double panX = panOffset.getX();
41            double panY = panOffset.getY();
42
43            // Przesunięcie względem pozycji kursora

```

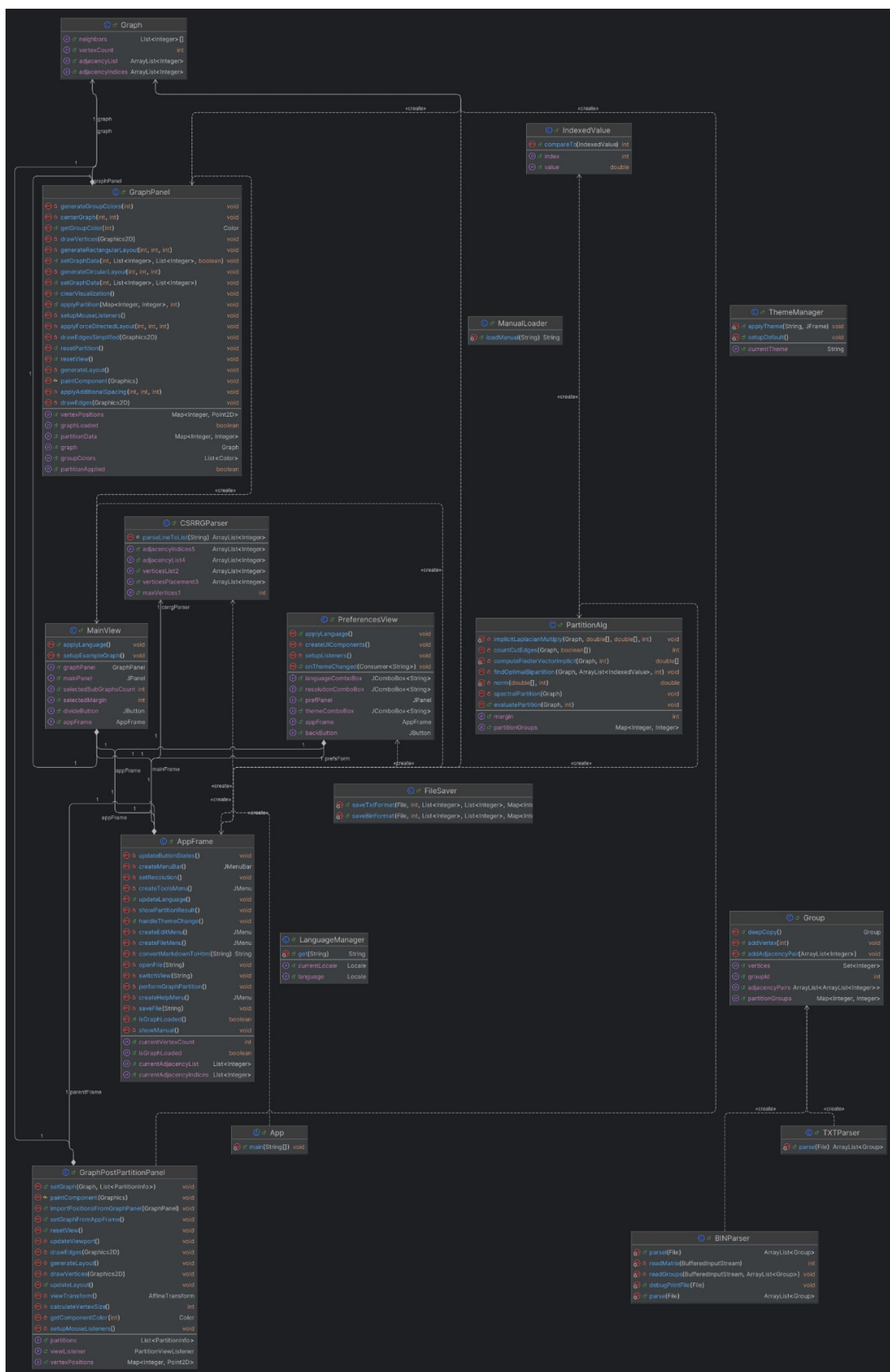
```

43         double newPanX = x - (x - panX) * zoomRatio;
44         double newPanY = y - (y - panY) * zoomRatio;
45
46         panOffset.setLocation(newPanX, newPanY);
47         zoomLevel = newZoom;
48
49         repaint();
50         updateViewport();
51     }
52 };
53
54 addMouseListener(ma);
55 addMouseMotionListener(ma);
56 addMouseWheelListener(ma);
57 }

```

Listing 11: Fragment kodu implementującego interaktywną nawigację po grafie

10 Diagram klas



Rys. 5: Diagram klas całego projektu.

11 Wzorce projektowe

W projekcie zostały zastosowane następujące wzorce projektowe:

- MVC (Model-View-Controller) - oddzielenie modelu danych grafu od jego wizualizacji i logiki sterującej
- Observer - powiadamianie komponentów GUI o zmianach w modelu danych
- Singleton - pojedyncze instancje menedżerów zasobów
- Adapter - dostosowanie formatu danych z pliku do modelu obiektowego

12 Instalacja i wymagania systemowe

Aplikacja została napisana w języku Java, co zapewnia jej przenośność między różnymi systemami operacyjnymi. Do uruchomienia i pracy z aplikacją wymagane jest:

- Java Development Kit (JDK) w wersji 17 lub nowszej
- Apache Maven 3.8 lub nowszy (przy kompilacji ze źródeł)
- Minimum 2GB RAM dla grafów 10^6 wierzchołków
- System operacyjny obsługujący Java Swing
- Rozdzielczość ekranu min. 1024x768

13 Podsumowanie i możliwości rozbudowy

- Aplikacja umożliwia szybkie testowanie podziału grafów, obsługuje różne języki, prezentuje instrukcję zgodnie z wybranym językiem, jest odporna na błędy plików wejściowych.
- Zaimplementowany algorytm spektralny zapewnia efektywny podział grafu z możliwością konfiguracji liczby części i marginesu błędu.
- Interaktywna wizualizacja z funkcjami nawigacji pozwala na wygodną analizę wyników.
- **Możliwości rozbudowy:**
 - Eksport wyników w dodatkowych formatach (np. do CSV, PNG, PDF)
 - Implementacja dodatkowych algorytmów podziału grafów (np. multi-level partitioning)
 - Rozbudowa możliwości edycji grafów bezpośrednio w aplikacji
 - Analityka wyników (statystyki, metryki efektywności podziału)
 - Integracja z bibliotekami grafowymi jak JGraphT
 - Rozbudowa instrukcji i samouczka dla użytkownika
 - Dodanie możliwości porównywania różnych strategii podziału